Self-Supervised Multi-Channel Hypergraph Convolutional Network for Social Recommendation

Junliang Yu
The University of Queensland
jl.yu@uq.edu.au

Hongzhi Yin*
The University of Queensland
h.yin1@uq.edu.au

Jundong Li
University of Virginia
jundong@virginia.edu

Qinyong Wang
The University of Queensland
qinyong.wang@uq.edu.au

Nguyen Quoc Viet Hung
Griffith University
quocviethung1@gmail.com

Xiangliang Zhang
King Abdullah University of Science and Technology
xiangliang.zhang@kaust.edu.sa

arXiv:2101.06448v4 [cs.IR] 27 Feb 2022

ABSTRACT

Social relations are often used to improve recommendation quality when user-item interaction data is sparse in recommender systems. Most existing social recommendation models exploit pairwise relations to mine potential user preferences. However, real-life interactions among users are very complex and user relations can be high-order. Hypergraph provides a natural way to model high-order relations, while its potentials for improving social recommendation are under-explored. In this paper, we fill this gap and propose a multi-channel hypergraph convolutional network to enhance social recommendation by leveraging high-order user relations. Technically, each channel in the network encodes a hypergraph that depicts a common high-order user relation pattern via hypergraph convolution. By aggregating the embeddings learned through multiple channels, we obtain comprehensive user representations to generate recommendation results. However, the aggregation operation might also obscure the inherent characteristics of different types of high-order connectivity information. To compensate for the aggregating loss, we innovatively integrate self-supervised learning into the training of the hypergraph convolutional network to regain the connectivity information with hierarchical mutual information maximization. Extensive experiments on multiple real-world datasets demonstrate the superiority of the proposed model over the current SOTA methods, and the ablation study verifies the effectiveness and rationale of the multi-channel setting and the self-supervised task. The implementation of our model is available at https://github.com/Coder-Yu/QRec.

CCS CONCEPTS

• **Information systems** → **Recommender systems**; **Social recommendation**.

KEYWORDS

Social Recommendation, Self-supervised Learning, Hypergraph Learning, Graph Convolutional Network, Recommender Systems

*Corresponding author and having equal contribution with the first author.

This paper is published under the Creative Commons Attribution 4.0 International (CC-BY 4.0) license. Authors reserve their rights to disseminate the work on their personal and corporate Web sites with the appropriate attribution.
WWW '21, April 19–23, 2021, Ljubljana, Slovenia
© 2021 IW3C2 (International World Wide Web Conference Committee), published under Creative Commons CC-BY 4.0 License.
ACM ISBN 978-1-4503-8312-7/21/04.
https://doi.org/10.1145/3442381.3449844

ACM Reference Format:
Junliang Yu, Hongzhi Yin, Jundong Li, Qinyong Wang, Nguyen Quoc Viet Hung, and Xiangliang Zhang. 2021. Self-Supervised Multi-Channel Hypergraph Convolutional Network for Social Recommendation. In *Proceedings of the Web Conference 2021 (WWW '21), April 19–23, 2021, Ljubljana, Slovenia.* ACM, New York, NY, USA, 11 pages. https://doi.org/10.1145/3442381.3449844

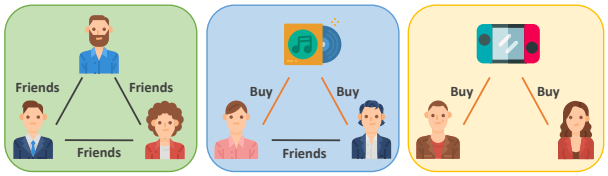

Figure 1: The common types of high-order user relations in social recommender systems.

1 INTRODUCTION

Over the past decade, the social media boom has dramatically changed people's ways of thinking and behaving. It has been revealed that people may alter their attitudes and behaviors in response to what they perceive their friends might do or think, which is known as the *social influence* [7]. Meanwhile, there are also studies [25] showing that people tend to build connections with others who have similar preferences with them, which is called the *homophily*. Based on these findings, social relations are often integrated into recommender systems to mitigate the data sparsity issue [13, 33]. Generally, in a social recommender system, if a user has few interactions with items, the system would rely on her friends' interactions to infer her preference and generate better recommendations. Upon this paradigm, a large number of social recommendation models have been developed [12, 21, 23, 55, 57, 61] and have shown stronger performance compared with general recommendation models.

Recently, graph neural networks (GNNs) [43] have achieved great success in a wide range of areas. Owing to their powerful capability in modeling relational data, GNNs-based models also have shown prominent performance in social recommendation [9, 19, 40–42, 58]. However, a key limitation of these GNNs-based social recommendation models is that they only exploit the simple pairwise user relations and ignore the ubiquitous high-order relations among users. Although the long-range dependencies of

relations (i.e. transitivity of friendship), which are also considered high-order, can be captured by using k graph neural layers to incorporate features from k-hop social neighbors, these GNNs-based models are unable to formulate and capture the complex high-order user relation patterns (as shown in Fig. 1) beyond pairwise relations. For example, it is natural to think that two users who are socially connected and also purchased the same item have a stronger relationship than those who are only socially connected, whereas the common purchase information in the former is often neglected in previous social recommendation models.

Hypergraph [4], which generalizes the concept of edge to make it connect more than two nodes, provides a natural way to model complex high-order relations among users. Despite the great advantages over the simple graph in user modeling, the strengths of hypergraph are under-explored in social recommendation. In this paper, we fill this gap by investigating the potentials of fusing hypergraph modeling and graph convolutional networks, and propose a **M**ulti-channel **H**ypergraph **C**onvolutional **N**etwork (**MHCN**) to enhance social recommendation by exploiting high-order user relations. Technically, we construct hypergraphs by unifying nodes that form specific triangular relations, which are instances of a set of carefully designed triangular motifs with underlying semantics (shown in Fig. 2). As we define multiple categories of motifs which concretize different types of high-order relations such as 'having a mutual friend', 'friends purchasing the same item', and 'strangers but purchasing the same item' in social recommender systems, each channel of the proposed hypergraph convolutional network undertakes the task of encoding a different motif-induced hypergraph. By aggregating multiple user embeddings learned through multiple channels, we can obtain the comprehensive user representations which are considered to contain multiple types of high-order relation information and have the great potentials to generate better recommendation results with the item embeddings.

However, despite the benefits of the multi-channel setting, the aggregation operation might also obscure the inherent characteristics of different types of high-order connectivity information [54], as different channels would learn embeddings with varying distributions on different hypergraphs. To address this issue and fully inherit the rich information in the hypergraphs, we innovatively integrate a self-supervised task [15, 37] into the training of the multi-channel hypergraph convolutional network. Unlike existing studies which enforce perturbations on graphs to augment the ground-truth [53], we propose to construct self-supervision signals by exploiting the hypergraph structures, with the intuition that the comprehensive user representation should reflect the user node's local and global high-order connectivity patterns in different hypergraphs. Concretely, we leverage the hierarchy in the hypergraph structures and hierarchically maximizes the mutual information between representations of the user, the user-centered sub-hypergraph, and the global hypergraph. The mutual information here measures the structural informativeness of the sub- and the whole hypergraph towards inferring the user features through the reduction in local and global structure uncertainty. Finally, we unify the recommendation task and the self-supervised task under a *primary & auxiliary* learning framework. By jointly optimizing the two tasks and leveraging the interplay of all the components, the performance of the recommendation task achieves significant gains.

The major contributions of this paper are summarized as follows:

- We investigate the potentials of fusing hypergraph modeling and graph neural networks in social recommendation by exploiting multiple types of high-order user relations under a multi-channel setting.
- We innovatively integrate self-supervised learning into the training of the hypergraph convolutional network and show that a self-supervised auxiliary task can significantly improve the social recommendation task.
- We conduct extensive experiments on multiple real-world datasets to demonstrate the superiority of the proposed model and thoroughly ablate the model to investigate the effectiveness of each component with an ablation study.

2 RELATED WORK

2.1 Social Recommendation

As suggested by the social science theories [7, 25], users' preferences and decisions are often influenced by their friends. Based on this fact, social relations are integrated into recommender systems to alleviate the issue of data sparsity. Early exploration of social recommender systems mostly focuses on matrix factorization (MF), which has a nice probabilistic interpretation with Gaussian prior and is the most used technique in social recommendation regime. The extensive use of MF marks a new phase in the research of recommender systems. A multitude of studies employ MF as their basic model to exploit social relations since it is very flexible for MF to incorporate prior knowledge. The common ideas of MF-based social recommendation algorithms can be categorized into three groups: co-factorization methods [22, 46], ensemble methods [20], and regularization methods [23]. Besides, there are also studies using socially-aware MF to model point-of-interest [48, 51, 52], preference evolution [39], item ranking [55, 61], and relation generation [11, 57].

Over the recent years, the boom of deep learning has broadened the ways to explore social recommendation. Many research efforts demonstrate that deep neural models are more capable of capturing high-level latent preferences [49, 50]. Specifically, graph neural networks (GNNs) [63] have achieved great success in this area, owing to their strong capability to model graph data. GraphRec [9] is the first to introduce GNNs to social recommendation by modeling the user-item and user-user interactions as graph data. DiffNet [41] and its extension DiffNet++ [40] model the recursive dynamic social diffusion in social recommendation with a layer-wise propagation structure. Wu *et al.* [42] propose a dual graph attention network to collaboratively learn representations for two-fold social effects. Song *et al.* develop DGRec [34] to model both users' session-based interests as well as dynamic social influences. Yu *et al.* [58] propose a deep adversarial framework based on GCNs to address the common issues in social recommendation. In summary, the common idea of these works is to model the user-user and user-item interactions as simple graphs with pairwise connections and then use multiple graph neural layers to capture the node dependencies.

2.2 Hypergraph in Recommender Systems

Hypergraph [4] provides a natural way to model complex high-order relations and has been extensively employed to tackle various

problems. With the development of deep learning, some studies combine GNNs and hypergraphs to enhance representation learning. HGNN [10] is the first work that designs a hypergraph convolution operation to handle complex data correlation in representation learning from a spectral perspective. Bai et al. [2] introduce hypergraph attention to hypergraph convolutional networks to improve their capacity. However, despite the great capacity in modeling complex data, the potentials of hypergraph for improving recommender systems have been rarely explored. There are only several studies focusing on the combination of these two topics. Bu *et al.* [5] introduce hypergraph learning to music recommender systems, which is the earliest attempt. The most recent combinations are HyperRec [38] and DHCF [16], which borrow the strengths of hypergraph neural networks to model the short-term user preference for next-item recommendation and the high-order correlations among users and items for general collaborative filtering, respectively. As for the applications in social recommendation, HMF [62] uses hypergraph topology to describe and analyze the interior relation of social network in recommender systems, but it does not fully exploit high-order social relations since HMF is a hybrid recommendation model. LBSN2Vec [47] is a social-aware POI recommendation model that builds hyperedges by jointly sampling friendships and check-ins with random walk, but it focuses on connecting different types of entities instead of exploiting the high-order social network structures.

2.3 Self-Supervised Learning

Self-supervised learning [15] is an emerging paradigm to learn with the ground-truth samples obtained from the raw data. It was firstly used in the image domain [1, 59] by rotating, cropping or colorizing the image to create auxiliary supervision signals. The latest advances in this area have extended self-supervised learning to graph representation learning [28, 29, 35, 37]. These studies mainly develop self-supervision tasks from the perspective of investigating graph structure. Node properties such as degree, proximity, and attributes, which are seen as local structure information, are often used as the ground truth to fully exploit the unlabeled data [17]. For example, InfoMotif [31] models attribute correlations in motif structures with mutual information maximization to regularize graph neural networks. Meanwhile, global structure information like node pair distance is also harnessed to facilitate representation learning [35]. Besides, contrasting congruent and incongruent views of graphs with mutual information maximization [29, 37] is another way to set up a self-supervised task, which has also shown promising results.

As the research of self-supervised learning is still in its infancy, there are only several works combining it with recommender systems [24, 44, 45, 64]. These efforts either mine self-supervision signals from future/surrounding sequential data [24, 45], or mask attributes of items/users to learn correlations of the raw data [64]. However, these thoughts cannot be easily adopted to social recommendation where temporal factors and attributes may not be available. The most relevant work to ours is GroupIM [32], which maximizes mutual information between representations of groups and group members to overcome the sparsity problem of group interactions. As the group can be seen as a special social clique, this work can be a corroboration of the effectiveness of social self-supervision signals.

3 PROPOSED MODEL

3.1 Preliminaries

Let $U = \{u_1, u_2, ..., u_m\}$ denote the user set ($|U| = m$), and $I = \{i_1, i_2, ..., i_n\}$ denote the item set ($|I| = n$). $\mathcal{I}(u)$ is the set of user consumption in which items consumed by user u are included. $\boldsymbol{R} \in \mathbb{R}^{m\times n}$ is a binary matrix that stores user-item interactions. For each pair (u, i), $r_{ui} = 1$ indicates that user u consumed item i while $r_{ui} = 0$ means that item i is unexposed to user u, or user u is not interested in item i. In this paper, we focus on top-K recommendation, and $\hat{r}_{ui}$ denotes the probability of item i to be recommended to user u. As for the social relations, we use $\boldsymbol{S} \in \mathbb{R}^{m\times m}$ to denote the relation matrix which is asymmetric because we work on directed social networks. In our model, we have multiple convolutional layers, and we use $\{\boldsymbol{P}^{(1)}, \boldsymbol{P}^{(2)}, \cdots, \boldsymbol{P}^{(l)}\} \in \mathbb{R}^{m\times d}$ and $\{\boldsymbol{Q}^{(1)}, \boldsymbol{Q}^{(2)}, \cdots, \boldsymbol{Q}^{(l)}\} \in \mathbb{R}^{n\times d}$ to denote the user and item embeddings of size d learned at each layer, respectively. In this paper, we use bold capital letters to denote matrices and bold lowercase letters to denote vectors.

Definition 1: Let $G = (V, E)$ denote a hypergraph, where V is the vertex set containing N unique vertices and E is the edge set containing M hyperedges. Each hyperedge $\epsilon \in E$ can contain any number of vertices and is assigned a positive weight $W_{\epsilon\epsilon}$, and all the weights formulate a diagonal matrix $\boldsymbol{W} \in \mathbb{R}^{M\times M}$. The hypergraph can be represented by an incidence matrix $\boldsymbol{H} \in \mathbb{R}^{N\times M}$ where $H_{i\epsilon} = 1$ if the hyperedge $\epsilon \in E$ contains a vertex $v_i \in V$, otherwise 0. The vertex and edge degree matrices are diagonal matrices denoted by $\boldsymbol{D}$ and $\boldsymbol{L}$, respectively, where $D_{ii} = \sum_{\epsilon=1}^{M} W_{\epsilon\epsilon}H_{i\epsilon}$; $L_{\epsilon\epsilon} = \sum_{i=1}^{N} H_{i\epsilon}$. It should be noted that, in this paper, $W_{\epsilon\epsilon}$ is uniformly assigned 1 and hence $\boldsymbol{W}$ is an identity matrix.

3.2 Multi-Channel Hypergraph Convolutional Network for Social Recommendation

In this section, we present our model **MHCN**, which stands for **M**ulti-channel **H**ypergraph **C**onvolutional **N**etwork. In Fig. 3, the schematic overview of our model is illustrated.

3.2.1 Hypergraph Construction. To formulate the high-order information among users, we first align the social network and user-item interaction graph in social recommender systems and then build hypergraphs over this heterogeneous network. Unlike prior models which construct hyperedges by unifying given types of entities [5, 47], our model constructs hyperedges according to the graph structure. As the relations in social networks are often directed, the connectivity of social networks can be of various types. In this paper, we use a set of carefully designed motifs to depict the common types of triangular structures in social networks, which guide the hypergraph construction.

Motif, as the specific local structure involving multiple nodes, is first introduced in [26]. It has been widely used to describe complex structures in a wide range of networks. In this paper, we only focus on triangular motifs because of the ubiquitous triadic closure in social networks, but our model can be seamlessly extended to handle on more complex motifs. Fig. 2 shows all the used triangular motifs. It has been revealed that $\mathcal{M}_1 - \mathcal{M}_7$ are crucial for

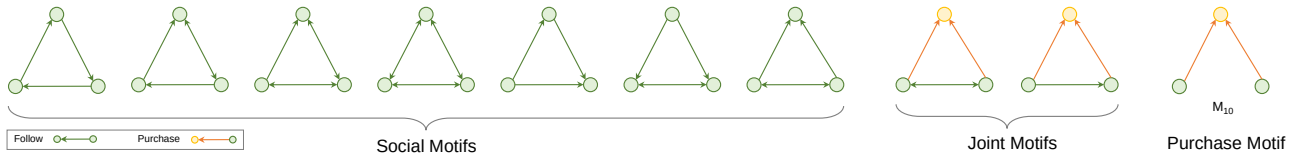

Figure 2: Triangle motifs used in our work. The green circles denote users and the yellow circles denote items.

social computing [3], and we further design $\mathcal{M}_8 - \mathcal{M}_{10}$ to involve user-item interactions to complement. Given motifs $\mathcal{M}_1 - \mathcal{M}_{10}$, we categorize them into three groups according to the underlying semantics. $\mathcal{M}_1 - \mathcal{M}_7$ summarize all the possible triangular relations in explicit social networks and describe the high-order social connectivity like 'having a mutual friend'. We name this group '*Social Motifs*'. $\mathcal{M}_8 - \mathcal{M}_9$ represent the compound relation, that is, 'friends purchasing the same item'. This type of relation can be seen as a signal of strengthened tie, and we name $\mathcal{M}_8 - \mathcal{M}_9$ '*Joint Motifs*'. Finally, we should also consider users who have no explicit social connections. So, $\mathcal{M}_{10}$ is non-closed and defines the implicit high-order social relation that users who are not socially connected but purchased the same item. We name $\mathcal{M}_{10}$ '*Purchase Motif*'. Under the regulation of these three types of motifs, we can construct three hypergraphs that contain different high-order user relation patterns. We use the incidence matrices $\boldsymbol{H}^s$, $\boldsymbol{H}^j$ and $\boldsymbol{H}^p$ to represent these three motif-induced hypergraphs, respectively, where each column of these matrices denotes a hyperedge. For example, in Fig. 3, $\{u_1, u_2, u_3\}$ is an instance of $\mathcal{M}_4$, and we use e_1 to denote this hyperedge. Then, according to definition 1, we have $H^s_{u_1,e_1} = H^s_{u_2,e_1} = H^s_{u_3,e_1} = 1$.

3.2.2 Multi-Channel Hypergraph Convolution. In this paper, we use a three-channel setting, including '*Social Channel* (s)', '*Joint Channel* (j)', and '*Purchase Channel* (p)', in response to the three types of triangular motifs, but the number of channels can be adjusted to adapt to more sophisticated situations. Each channel is responsible for encoding one type of high-order user relation pattern. As different patterns may show different importances to the final recommendation performance, directly feeding the full base user embeddings $\boldsymbol{P}^{(0)}$ to all the channels is unwise. To control the information flow from the base user embeddings $\boldsymbol{P}^{(0)}$ to each channel, we design a pre-filter with *self-gating units* (SGUs), which is defined as:

$$\boldsymbol{P}_c^{(0)} = f^c_{\text{gate}}(\boldsymbol{P}^{(0)}) = \boldsymbol{P}^{(0)} \odot \sigma(\boldsymbol{P}^{(0)}\boldsymbol{W}_g^c + \boldsymbol{b}_g^c), \tag{1}$$

where $\boldsymbol{W}_g^c \in \mathbb{R}^{d\times d}, \boldsymbol{b}_g^c \in \mathbb{R}^d$ are parameters to be learned, $c \in \{s, j, p\}$ represents the channel, $\odot$ denotes the element-wise product and σ is the sigmoid nonlinearity. The self-gating mechanism effectively serves as a multiplicative skip-connection [8] that learns a nonlinear gate to modulate the base user embeddings at a feature-wise granularity through dimension re-weighting, then we obtain the channel-specific user embeddings $\boldsymbol{P}_c^{(0)}$.

如何理解这里的feature-wise embeddings

Referring to the spectral hypergraph convolution proposed in [10], we define our hypergraph convolution as:

$$\boldsymbol{P}_c^{(l+1)} = \boldsymbol{D}_c^{-1}\boldsymbol{H}_c\boldsymbol{L}_c^{-1}\boldsymbol{H}_c^\top\boldsymbol{P}_c^{(l)}. \tag{2}$$

为什么这么定义？

属于同一条 hyperedge 的节点信息聚到超边上
再用 Hc 把超边信息回灌给节点

Table 1: Computation of motif-induced adjacency matrices.

Motif	Matrix Computation	$\boldsymbol{A}_{M_i} =$
$\mathcal{M}_1$	$\boldsymbol{C} = (\boldsymbol{U}\boldsymbol{U}) \odot \boldsymbol{U}^T$	$\boldsymbol{C} + \boldsymbol{C}^\top$
$\mathcal{M}_2$	$\boldsymbol{C} = (\boldsymbol{B}\boldsymbol{U}) \odot \boldsymbol{U}^T + (\boldsymbol{U}\boldsymbol{B}) \odot \boldsymbol{U}^T + (\boldsymbol{U}\boldsymbol{U}) \odot \boldsymbol{B}$	$\boldsymbol{C} + \boldsymbol{C}^\top$
$\mathcal{M}_3$	$\boldsymbol{C} = (\boldsymbol{B}\boldsymbol{B}) \odot \boldsymbol{U} + (\boldsymbol{B}\boldsymbol{U}) \odot \boldsymbol{B} + (\boldsymbol{U} \cdot \boldsymbol{B}) \odot \boldsymbol{B}$	$\boldsymbol{C} + \boldsymbol{C}^\top$
$\mathcal{M}_4$	$\boldsymbol{C} = (\boldsymbol{B}\boldsymbol{B}) \odot \boldsymbol{B}$	$\boldsymbol{C}$
$\mathcal{M}_5$	$\boldsymbol{C} = (\boldsymbol{U}\boldsymbol{U}) \odot \boldsymbol{U} + (\boldsymbol{U}\boldsymbol{U}^T) \odot \boldsymbol{U} + (\boldsymbol{U}^T\boldsymbol{U}) \odot \boldsymbol{U}$	$\boldsymbol{C} + \boldsymbol{C}^\top$
$\mathcal{M}_6$	$\boldsymbol{C} = (\boldsymbol{U}\boldsymbol{B}) \odot \boldsymbol{U} + (\boldsymbol{B}\boldsymbol{U}^T) \odot \boldsymbol{U}^T + (\boldsymbol{U}^T\boldsymbol{U}) \odot \boldsymbol{B}$	$\boldsymbol{C}$
$\mathcal{M}_7$	$\boldsymbol{C} = (\boldsymbol{U}^T\boldsymbol{B}) \odot \boldsymbol{U}^T + (\boldsymbol{B}\boldsymbol{U}) \odot \boldsymbol{U} + (\boldsymbol{U}\boldsymbol{U}^T) \odot \boldsymbol{B}$	$\boldsymbol{C}$
$\mathcal{M}_8$	$\boldsymbol{C} = (\boldsymbol{R}\boldsymbol{R}^T) \odot \boldsymbol{B}$	$\boldsymbol{C}$
$\mathcal{M}_9$	$\boldsymbol{C} = (\boldsymbol{R}\boldsymbol{R}^T) \odot \boldsymbol{U}$	$\boldsymbol{C} + \boldsymbol{C}^\top$
$\mathcal{M}_{10}$	$\boldsymbol{C} = \boldsymbol{R}\boldsymbol{R}^T$	$\boldsymbol{C}$

The difference is that we follow the suggestion in [6, 14] to remove the learnable matrix for linear transformation and the nonlinear activation function (e.g. leaky ReLU). By replacing $\boldsymbol{H}_c$ with any of $\boldsymbol{H}^s$, $\boldsymbol{H}^j$ and $\boldsymbol{H}^p$, we can borrow the strengths of hypergraph convolutional networks to learn user representations encoded high-order information in the corresponding channel. As $\boldsymbol{D}_c$ and $\boldsymbol{L}_c$ are diagonal matrices which only re-scale embeddings, we skip them in the following discussion. The hypergraph convolution can be viewed as a two-stage refinement performing 'node-hyperedge-node' feature transformation upon hypergraph structure. The multiplication operation $\boldsymbol{H}_c^\top\boldsymbol{P}_c^{(l)}$ defines the message passing from nodes to hyperedges and then premultiplying $\boldsymbol{H}_c$ is viewed to aggregate information from hyperedges to nodes. However, despite the benefits of hypergraph convolution, there are a huge number of motif-induced hyperedges (e.g. there are 19,385 social triangles in the used dataset, LastFM), which would cause a high cost to build the incidence matrix $\boldsymbol{H}_c$. But as we only exploit triangular motifs, we show that this problem can be solved in a flexible and efficient way by leveraging the associative property of matrix multiplication.

显式列出所有hyperedges 很贵。因为motif三角实例可能非常多

Following [60], we let $\boldsymbol{B} = \boldsymbol{S} \odot \boldsymbol{S}^T$ and $\boldsymbol{U} = \boldsymbol{S} - \boldsymbol{B}$ be the adjacency matrices of the bidirectional and unidirectional social networks respectively. We use $\boldsymbol{A}_{M_k}$ to represent the motif-induced adjacency matrix and $(\boldsymbol{A}_{M_k})_{i,j} = 1$ means that vertex i and vertex j appear in one instance of $\mathcal{M}_k$. As two vertices can appear in multiple instances of $\mathcal{M}_k$, $(\boldsymbol{A}_{M_k})_{i,j}$ is computed by:

$$(\boldsymbol{A}_{M_k})_{i,j} = \#(i, j \text{ occur in the same instance of } \mathcal{M}_k). \tag{3}$$

Table 1 shows how to calculate $\boldsymbol{A}_{M_k}$ in the form of matrix multiplication. As all the involved matrices in Table 1 are sparse matrices, $\boldsymbol{A}_{M_k}$ can be efficiently calculated. Specifically, the basic unit in Table 1 is in a general form of $\boldsymbol{X}\boldsymbol{Y} \odot \boldsymbol{Z}$, which means $\boldsymbol{A}_{M_1}$ to $\boldsymbol{A}_{M_9}$ may be sparser than $\boldsymbol{Z}$ (i.e. $\boldsymbol{B}$ or $\boldsymbol{U}$) or as sparse as $\boldsymbol{Z}$. $\boldsymbol{A}_{M_{10}}$ could be a little denser, but we can filter out the popular items (we think consuming

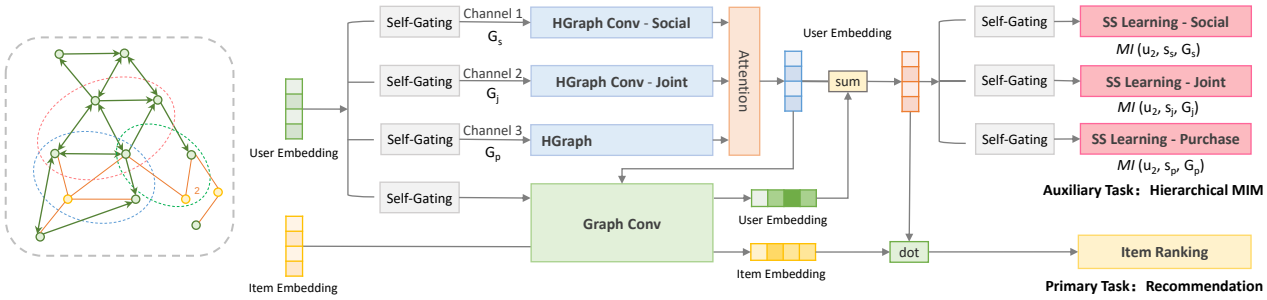

Figure 3: An overview of the proposed model (1-layer). Each triangle in the left graph is a hyperedge and also an instance of defined motifs. G_s, G_j and G_p denote the three motif-induced hypergraphs constructed based on social, joint, and purchase motifs, respectively. s_s, s_j, and s_p in the three dotted ellipses denote three ego-networks with u_2 as the center, which are subgraphs of G_s, G_j and G_p, respectively.

两个用户在当前channel的motif实例里一同出现了多少次

popular items might not reflect the users' personalized preferences) when calculating $\boldsymbol{A}_{M_{10}}$ and remove the entries less than a threshold (e.g. 5) in $\boldsymbol{A}_{M_{10}}$ to keep efficient calculation. For symmetric motifs, $\boldsymbol{A}_M = \boldsymbol{C}$, and for the asymmetric ones $\boldsymbol{A}_M = \boldsymbol{C} + \boldsymbol{C}^T$. Obviously, without considering self-connection, the summation of $\boldsymbol{A}_{M_1}$ to $\boldsymbol{A}_{M_7}$ is equal to $\boldsymbol{H}^s\boldsymbol{H}^{s\top}$, as each entry of $\boldsymbol{H}^s\boldsymbol{H}^{s\top} \in \mathbb{R}^{m\times m}$ also indicates how many social triangles contain the node pair represented by the row and column index of the entry. Analogously, the summation of $\boldsymbol{A}_{M_8}$ to $\boldsymbol{A}_{M_9}$ is equal to $\boldsymbol{H}^j\boldsymbol{H}^{j\top}$ without self-connection and $\boldsymbol{A}_{M_{10}}$ is equal to $\boldsymbol{H}^p\boldsymbol{H}^{p\top}$. Taking the calculation of $\boldsymbol{A}_{M_1}$ as an example, it is evident that $\boldsymbol{UU}$ constructs a unidirectional path connecting three vertices, and the operation $\odot\boldsymbol{U}$ makes the path into a loop, which is an instance of $\boldsymbol{A}_{M_1}$. As $\boldsymbol{A}_{M_{10}}$ also contains the triangles in $\boldsymbol{A}_{M_8}$ and $\boldsymbol{A}_{M_9}$. So, we remove the redundancy from $\boldsymbol{A}_{M_{10}}$. Finally, we use $\boldsymbol{A}_s = \sum_{k=1}^{7} \boldsymbol{A}_{M_k}$, $\boldsymbol{A}_j = \boldsymbol{A}_{M_8} + \boldsymbol{A}_{M_9}$, and $\boldsymbol{A}_p = \boldsymbol{A}_{M_{10}} - \boldsymbol{A}_j$ to replace $\boldsymbol{H}^s\boldsymbol{H}^{s\top}$, $\boldsymbol{H}^j\boldsymbol{H}^{j\top}$, and $\boldsymbol{H}^p\boldsymbol{H}^{p\top}$ in Eq. (2), respectively. Then we have a transformed hypergraph convolution, defined as:

$$\boldsymbol{P}_c^{(l+1)} = \hat{\boldsymbol{D}}_c^{-1}\boldsymbol{A}_c\boldsymbol{P}_c^{(l)}, \tag{4}$$

where $\hat{\boldsymbol{D}}_c \in \mathbb{R}^{m\times m}$ is the degree matrix of $\boldsymbol{A}_c$. Obviously, Eq (4) is equivalent to Eq (2), and can be a simplified substitution of the hypergraph convolution. Since we follow the design of LightGCN which has subsumed the effect of self-connection, and thus skipping self-connection in adjacency matrix does not matter too much. In this way, we circumvent the individual hyperedge construction and computation, and greatly reduce the computational cost.

3.2.3 Learning Comprehensive User Representations. After propagating the user embeddings through L layers, we average the embeddings obtained at each layer to form the final channel-specific user representation: $\boldsymbol{P}_c^* = \frac{1}{L+1}\sum_{l=0}^{L}\boldsymbol{P}_c^{(l)}$ to avoid the over-smoothing problem [14]. Then we use the attention mechanism [36] to selectively aggregate information from different channel-specific user embeddings to form the comprehensive user embeddings. For each user u, a triplet $(\alpha^s, \alpha^j, \alpha^p)$ is learned to measure the different contributions of the three channel-specific embeddings to the final recommendation performance. The attention function f_{att} is defined as:

$$\alpha_c = f_{\text{att}}(p_c^*) = \frac{\exp(\boldsymbol{a}^\top \cdot \boldsymbol{W}_{att}\boldsymbol{p}_c^*)}{\sum_{c'\in\{s,j,p\}}\exp(\boldsymbol{a}^\top \cdot \boldsymbol{W}_{att}\boldsymbol{p}_{c'}^*)}, \tag{5}$$

where $\boldsymbol{a} \in \mathbb{R}^d$ and $\boldsymbol{W}_{att} \in \mathbb{R}^{d\times d}$ are trainable parameters, and the comprehensive user representation $\boldsymbol{p}^* = \sum_{c\in\{s,j,p\}}\alpha_c\boldsymbol{p}_c^*$.

Note that, since the explicit social relations are noisy and isolated relations are not a strong signal of close friendship [55, 56], we discard those relations which are not part of any instance of defined motifs. So, we do not have a convolution operation directly working on the explicit social network $\boldsymbol{S}$. Besides, in our setting, the hypergraph convolution cannot directly aggregate information from the items (we do not incorporate the items into $\boldsymbol{A}_j$ and $\boldsymbol{A}_p$). To tackle this problem, we additionally perform simple graph convolution on the user-item interaction graph to encode the purchase information and complement the multi-channel hypergraph convolution. The simple graph convolution is defined as:

$$\begin{aligned}
\boldsymbol{P}_r^{(l+1)} &= \boldsymbol{D}_u^{-1}\boldsymbol{R}\boldsymbol{Q}^{(l)}, \quad \boldsymbol{P}_r^{(0)} = f_{\text{gate}}^r(\boldsymbol{P}^{(0)}),\\
\boldsymbol{Q}^{(l+1)} &= \boldsymbol{D}_i^{-1}\underbrace{\boldsymbol{R}^\top}_{\substack{\text{user-item}\\ \text{interactions}\\ \text{matrix}}}\boldsymbol{P}_m^{(l)}, \quad \boldsymbol{P}_m^{(l)} = \sum_{c\in\{s,j,p\}}\alpha_c\boldsymbol{p}_c^{(l)} + \frac{1}{2}\boldsymbol{P}_r^{(l)},
\end{aligned} \tag{6}$$

where $\boldsymbol{P}_r^{(l)}$ is the gated user embeddings for simple graph convolution, $\boldsymbol{P}_m^{(l)}$ is the combination of the comprehensive user embeddings and $\boldsymbol{P}_r^{(l)}$, and $\boldsymbol{D}_u \in \mathbb{R}^{m\times m}$ and $\boldsymbol{D}_i \in \mathbb{R}^{n\times n}$ are degree matrices of $\boldsymbol{R}$ and $\boldsymbol{R}^\top$, respectively. Finally, we obtain the final user and item embeddings $\boldsymbol{P}$ and $\boldsymbol{Q}$ defined as:

$$\boldsymbol{P} = \boldsymbol{P}^* + \frac{1}{L+1}\sum_{l=0}^{L}\boldsymbol{P}_r^{(l)}, \quad \boldsymbol{Q} = \frac{1}{L+1}\sum_{l=0}^{L}\boldsymbol{Q}^{(l)}, \tag{7}$$

where $\boldsymbol{P}^{(0)}$ and $\boldsymbol{Q}^{(0)}$ are randomly initialized.

3.2.4 Model Optimization. To learn the parameters of MHCN, we employ the Bayesian Personalized Ranking (BPR) loss [30], which is a pairwise loss that promotes an observed entry to be ranked higher than its unobserved counterparts:

$$\mathcal{L}_r = \sum_{i\in\mathcal{I}(u), j\notin\mathcal{I}(u)} -\log\sigma\left(\hat{r}_{u,i}(\Phi) - \hat{r}_{u,j}(\Phi)\right) + \lambda\|\Phi\|_2^2, \tag{8}$$

where Φ denotes the parameters of MHCN, $\hat{r}_{u,i} = \boldsymbol{p}_u^\top\boldsymbol{q}_i$ is the predicted score of u on i, and $\sigma(\cdot)$ here is the sigmoid function. Each time a triplet including the current user u, the positive item i purchased by u, and the randomly sampled negative item j which is disliked by u or unknown to u, is fed to MHCN. The model is

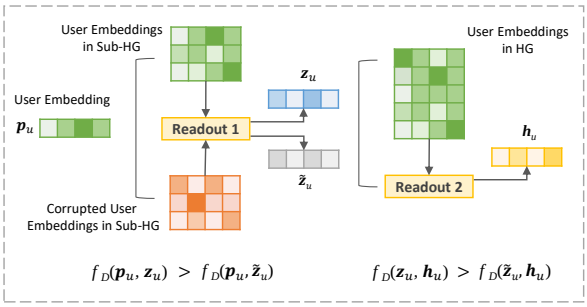

Figure 4: Hierarchical mutual information maximization on hypergraphs.

optimized towards ranking i higher than j in the recommendation list for u. In addition, L_2 regularization with the hyper-parameter λ is imposed to reduce generalized errors.

3.3 Enhancing MHCN with Self-Supervised Learning

Owing to the exploitation of high-order relations, MHCN shows great performance (reported in Table 3 and 4). However, a shortcoming of MHCN is that the aggregation operations (Eq. 5 and 6) might lead to a loss of high-order information, as different channels would learn embeddings with varying distributions on different hypergraphs [54]. Concatenating the embeddings from different channels could be the alternative, but it uniformly weighs the contributions of different types of high-order information in recommendation generation, which is not in line with the reality and leads to inferior performance in our trials. To address this issue and fully inherit the rich information in the hypergraphs, we innovatively integrate self-supervised learning into the training of MHCN.

In the scenarios of representation learning, self-supervised task usually either serves as a pretraining strategy or an auxiliary task to improve the primary task [17]. In this paper, we follow the *primary & auxiliary* paradigm, and set up a self-supervised auxiliary task to enhance the recommendation task (primary task). The recent work *Deep Graph Infomax* (DGI) [37] is a general and popular approach for learning node representations within graph-structured data in a self-supervised manner. It relies on maximizing mutual information (MI) between node representations and corresponding high-level summaries of graphs. However, we consider that the graph-node MI maximization stays at a coarse level and there is no guarantee that the encoder in DGI can distill sufficient information from the input data. Therefore, with the increase of the graph scale, the benefits brought by MI maximization might diminish. For a better learning method which fits our scenario more, we inherit the merits of DGI to consider mutual information and further extend the graph-node MI maximization to a fine-grained level by exploiting the hierarchical structure in hypergraphs.

Recall that, for each channel of MHCN, we build the adjacency matrix $\boldsymbol{A}_c$ to capture the high-order connectivity information. Each row in $\boldsymbol{A}_c$ represents a subgraph of the corresponding hypergraph centering around the user denoted by the row index. Then we can induce a hierarchy: 'user node ← user-centered sub-hypergraph ← hypergraph' and create self-supervision signals from this structure. Our intuition of the self-supervised task is that the comprehensive user representation should reflect the user node's local and global high-order connectivity patterns in different hypergraphs, and this goal can be achieved by hierarchically maximizing the mutual information between representations of the user, the user-centered sub-hypergraph, and the hypergraph in each channel. The mutual information measures the structural informativeness of the sub- and the whole hypergraph towards inferring the user preference through the reduction in local and global structure uncertainty.

To get the sub-hypergraph representation, instead of averaging the embeddings of the users in the sub-hypergraph, we design a readout function $f_{\text{out}_1} : \mathbb{R}^{k\times d} \rightarrow \mathbb{R}^d$, which is permutation-invariant and formulated as:

$$z_u^c = f_{out_1}(\boldsymbol{P}_c, \boldsymbol{a}_u^c) = \frac{\boldsymbol{P}_c \boldsymbol{a}_u^c}{sum(\boldsymbol{a}_u^c)}, \tag{9}$$

where $\boldsymbol{P}_c = f_{\text{gate}}^c(\boldsymbol{P})$ is to control the participated magnitude of $\boldsymbol{P}$ to avoid overfitting and mitigate gradient conflict between the primary and auxiliary tasks, $\boldsymbol{a}_u^c$ is the row vector of $\boldsymbol{A}_c$ corresponding to the center user u, and $sum(\boldsymbol{a}_u^c)$ denotes how many connections in the sub-hypergraph. In this way, the weight (importance) of each user in the sub-hypergraph is considered to form the sub-hypergraph embedding $\boldsymbol{z}_u$. Analogously, we define the other readout function $f_{\text{out}_2} : \mathbb{R}^{m\times d} \rightarrow \mathbb{R}^d$, which is actually an average pooling to summarize the obtained sub-hypergraph embeddings into a graph-level representation:

$$\boldsymbol{h}^c = f_{out_2}(\boldsymbol{Z}_c) = \text{AveragePooling}(\boldsymbol{Z}_c). \tag{10}$$

We tried to use InfoNCE [27] as our learning objective to maximize the hierarchical mutual information. But we find that the pairwise ranking loss, which has also been proved to be effective in mutual information estimation [18], is more compatible with the recommendation task. We then define the objective function of the self-supervised task as follows:

$$\begin{aligned}\mathcal{L}_s = -\sum_{c\in\{s,j,p\}} \Big\{ &\sum_{u\in U} \log\sigma(f_D(\boldsymbol{p}_u^c, \boldsymbol{z}_u^c) - f_D(\boldsymbol{p}_u^c, \tilde{\boldsymbol{z}}_u^c)) \\ &+ \sum_{u\in U} \log\sigma(f_D(\boldsymbol{z}_u^c, \boldsymbol{h}^c) - f_D(\tilde{\boldsymbol{z}}_u^c, \boldsymbol{h}^c))\Big\}.\end{aligned} \tag{11}$$

$f_D(\cdot) : \mathbb{R}^d \times \mathbb{R}^d \longmapsto \mathbb{R}$ is the discriminator function that takes two vectors as the input and then scores the agreement between them. We simply implement the discriminator as the dot product between two representations. Since there is a bijective mapping between $\boldsymbol{P}_c$ and $\boldsymbol{Z}_c$, they can be the ground truth of each other. We corrupt $\boldsymbol{Z}_c$ by both row-wise and column-wise shuffling to create negative examples $\tilde{\boldsymbol{Z}}_c$. We consider that, the user should have a stronger connection with the sub-hypergraph centered with her (local structure), so we directly maximize the mutual information between their representations. By contrast, the user would not care all the other users too much (global structure), so we indirectly maximize the mutual information between the representations of the user and the complete hypergraph by regarding the sub-hypergraph as the mediator. Compared with DGI which only maximizes the mutual information between node and graph representations, our hierarchical design can preserve more structural information of the hypergraph into the user representations (comparison is shown in

Section 4.3). Figure 4 illustrates the hierarchical mutual information maximization.

Finally, we unify the objectives of the recommendation task (primary) and the task of maximizing hierarchical mutual information (auxiliary) for joint learning. The overall objective is defined as:

$$\mathcal{L} = \mathcal{L}_r + \beta \mathcal{L}_s, \tag{12}$$

where β is a hyper-parameter used to control the effect of the auxiliary task and $\mathcal{L}_s$ can be seen as a regularizer leveraging hierarchical structural information of the hypergraphs to enrich the user representations in the recommendation task for a better performance.

3.4 Complexity Analysis

In this section, we discuss the complexity of our model.

Model size. The trainable parameters of our model consist of three parts: user and item embeddings, gate parameters, and attention parameters. For the first term, we only need to learn the 0th layer user embeddings $\boldsymbol{P}^{(0)} \in \mathbb{R}^{m \times d}$ and item embeddings $\boldsymbol{Q}^{(0)} \in \mathbb{R}^{n \times d}$. As for the second term, we employ seven gates, four for MHCN and three for the self-supervised task. Each of the gate has parameters of size $(d+1) \times d$, while the attention parameters are of the same size. To sum up, the model size approximates $(m + n + 8d)d$ in total. As $\min(m, n) \gg d$, our model is fairly light.

Time complexity. The computational cost mainly derives from four parts: hypergraph/graph convolution, attention, self-gating, and mutual information maximization. For the multi-channel hypergraph convolution through L layers, the propagation consumption is less than $O(|\boldsymbol{A}^+|dL)$, where $|\boldsymbol{A}^+|$ denotes the number of nonzero elements in $\boldsymbol{A}$, and here $|\boldsymbol{A}^+| = \max(|\boldsymbol{A}_s^+|, |\boldsymbol{A}_j^+|, |\boldsymbol{A}_p^+|)$. Analogously, the time complexity of the graph convolution is $O(|\boldsymbol{R}^+|dL)$. As for the attention and self gating mechanism, they both contribute $O(md^2)$ time complexity. The cost of mutual information maximization is mainly from f_{out_1}, which is $O(|\boldsymbol{A}^+|d)$. Since we follow the setting in [14] to remove the learnable matrix for linear transformation and the nonlinear activation function, the time complexity of our model is much lower than that of previous GNNs-based social recommendation models.

Table 2: Dataset Statistics

Dataset	#User	#Item	#Feedback	#Relation	Density
LastFM	1,892	17,632	92,834	25,434	0.28%
Douban	2,848	39,586	894,887	35,770	0.79%
Yelp	19,539	21,266	450,884	864,157	0.11%

4 EXPERIMENTS AND RESULTS

4.1 Experimental Settings

Datasets. Three real-world datasets: LastFM[1], Douban[2], and Yelp [49] are used in our experiments. As our aim is to generate Top-K recommendation, for Douban which is based on explicit ratings, we leave out ratings less than 4 and assign 1 to the rest. The statistics of the datasets is shown in Table 2. We perform 5-fold cross-validation on the three datasets and report the average results.

Baselines. We compare MHCN with a set of strong and commonly-used baselines including MF-based and GNN-based models:

- **BPR** [30] is a popular recommendation model based on Bayesian personalized ranking. It models the order of candidate items by a pairwise ranking loss.
- **SBPR** [61] is a MF based social recommendation model which extends BPR and leverages social connections to model the relative order of candidate items.
- **LightGCN** [14] is an efficient GCN-based general recommendation model that leverages the user-item proximity to learn node representations and generate recommendations.
- **GraphRec** [9] is the first GNN-based social recommendation model that models both user-item and user-user interactions.
- **DiffNet++** [40] is the latest GCN-based social recommendation method that models the recursive dynamic social diffusion in both the user and item spaces.
- **DHCF** [16] is a recent hypergraph convolutional network-based method that models the high-order correlations among users and items for general recommendation.

Two versions of the proposed multi-channel hypergraph convolutional network are investigated in the experiments. **MHCN** denotes the vanilla version and **S^2-MHCN** denotes the self-supervised version.

Metrics. To evaluate the performance of all methods, two relevancy-based metrics *Precision@10* and *Recall@10* and one ranking-based metric *NDCG@10* are used. We perform item ranking on all the candidate items instead of the sampled item sets to calculate the values of these three metrics, which guarantees that the evaluation process is unbiased.

Settings. For a fair comparison, we refer to the best parameter settings reported in the original papers of the baselines and then use grid search to fine tune all the hyperparameters of the baselines to ensure the best performance of them. For the general settings of all the models, the dimension of latent factors (embeddings) is empirically set to 50, the regularization coefficient $\lambda = 0.001$, and the batch size is set to 2000. We use Adam to optimize all these models. Section 4.4 reports the influence of different parameters (i.e. β and the depth) of MHCN, and we use the best parameter settings in Section 4.2, and 4.3.

4.2 Recommendation Performance

In this part, we validate if MHCN outperforms existing social recommendation baselines. Since the primary goal of social recommendation is to mitigate data sparsity issue and improve the recommendation performance for cold-start users. Therefore, we respectively conduct experiments on the complete test set and the cold-start test set in which only the cold-start users with less than 20 interactions are contained. The experimental results are shown in Table 3 and Table 4. The improvement is calculated by subtracting the best performance value of the baselines from that of S^2-MHCN and then using the difference to divide the former. Analogously, S^2-improvement is calculated by comparing the values of the performance of MHCN and and S^2-MHCN. According to the results, we can draw the following conclusions:

[1] http://files.grouplens.org/datasets/hetrec2011/
[2] https://pan.baidu.com/s/1hrJP6rq

Table 3: General recommendation performance comparison.

Dataset	Metric	GraphRec	BPR	SBPR	DiffNet++	DHCF	LightGCN	**MHCN**	**S^2-MHCN**	Improv.	S^2-Improv.
LastFM	P@10	17.385%	16.556%	16.893%	18.485%	16.877%	19.205%	19.625%	**20.052%**	4.410%	2.175%
	R@10	18.020%	16.803%	17.245%	18.737%	17.131%	19.480%	19.945%	**20.375%**	4.594%	2.155%
	N@10	0.21173	0.19943	0.20516	0.22310	0.20744	0.23392	0.23834	**0.24395**	4.287%	2.156%
Douban	P@10	17.021%	15.673%	15.993%	17.532%	16.871%	17.780%	18.283%	**18.506%**	4.083%	1.220%
	R@10	5.916%	5.160%	5.322%	6.205%	5.755%	6.247%	6.556%	**6.681%**	6.947%	1.906%
	N@10	0.19051	0.17476	0.17821	0.19701	0.18655	0.19881	0.20694	**0.21038**	5.819%	1.662%
Yelp	P@10	2.323%	2.002%	2.192%	2.480%	2.298%	2.586%	2.751%	**3.003%**	16.125%	9.160%
	R@10	6.075%	5.173%	5.468%	6.354%	5.986%	6.525%	6.862%	**7.885%**	17.247%	14.908%
	N@10	0.04653	0.03840	0.04314	0.04833	0.04700	0.04998	0.05356	**0.06061**	21.268%	13.162%

Table 4: Cold-start recommendation performance comparison.

Dataset	Metric	GraphRec	BPR	SBPR	DiffNet++	DHCF	LightGCN	**MHCN**	**S^2-MHCN**	Improv.	S^2-Improv.
LastFM	P@10	4.662%	3.784%	4.573%	5.102%	3.974%	4.809%	5.466%	**5.759%**	12.877%	5.360%
	R@10	18.033%	15.240%	18.417%	21.365%	16.395%	20.361%	23.354%	**24.431%**	14.350%	4.611%
	N@10	0.14675	0.12460	0.15141	0.16031	0.14285	0.15044	0.17218	**0.19138**	19.381%	11.151%
Douban	P@10	2.007%	1.722%	1.935%	2.230%	1.921%	2.134%	2.343%	**2.393%**	7.309%	2.133%
	R@10	8.215%	7.178%	8.084%	8.705%	7.977%	8.317%	9.646%	**10.632%**	22.136%	10.227%
	N@10	0.05887	0.04784	0.05716	0.06767	0.05533	0.06037	0.06771	**0.07113**	5.113%	5.052%
Yelp	P@10	1.355%	1.232%	1.286%	1.475%	1.314%	1.504%	1.545%	**1.747%**	14.108%	13.074%
	R@10	5.901%	5.468%	5.720%	6.635%	5.876%	6.753%	6.838%	**7.881%**	12.264%	15.253%
	N@10	0.03896	0.03448	0.03671	0.04237	0.03826	0.04273	0.04354	**0.05143**	15.703%	18.121%

- MHCN shows great performance in both the general and cold-start recommendation tasks. Even without self-supervised learning, it beats all the baselines by a fair margin. Meanwhile, self-supervised learning has great ability to further improve MHCN. Compared with the vanilla version, the self-supervised version shows decent improvements in all the cases. Particularly, in the cold-start recommendation task, self-supervised learning brings significant gains. On average, S^2-MHCN achieves about 5.389% improvement in the general recommendation task and 9.442% improvement in the cold-start recommendation task compared with MHCN. Besides, it seems that, the sparser the dataset, the more improvements self-supervised learning brings.
- GNN-based recommendation models significantly outperform the MF-based recommendation models. Even the general recommendation models based on GNNs show much better performance than MF-based social recommendation models. However, when compared with the counterparts based on the same building block (i.e. MF-based *vs.* MF-based, GNNs-based *vs.* GNNs-based), social recommendation models are still competitive and by and large outperform the corresponding general recommendation models except LightGCN.
- LightGCN is a very strong baseline. Without considering the two variants of MHCN, LightGCN shows the best or the second best performance in most cases. This can be owed to the removal of the redundant operations including the nonlinear activation function and transformation matrices. The other baselines such as GraphRec might be limited by these useless operations, and fail to outperform LightGCN, though the social information is incorporated.
- Although DHCF is also based on hypergraph convolution, it does not show any competence in all the cases. We are unable to reproduce its superiority reported in the original paper [16]. There are two possible causes which might lead to its failure. Firstly, it only exploits the user-item high-order relations. Secondly, the way to construct hyperedges is very impractical in this model, which leads to a very dense incidence matrix. The model would then encounter the over-smoothing problem and suffer from heavy computation.

4.3 Ablation Study

In this section, we conduct an ablation study to investigate the interplay of the components in S^2-MHCN and validate if each component positively contributes to the final recommendation performance.

4.3.1 Investigation of Multi-Channel Setting. We first investigate the multi-channel setting by removing any of the three channels from S^2-MHCN and leaving the other two to observe the changes of performance. Each bar in the plots (except *complete*) represents the case that the corresponding channel is removed, while *complete* means no module has been removed. From Fig. 5, we can observe that removing any channel would cause performance degradation. But it is obvious that *purchase channel* contributes the most to the final performance. Without this channel, S^2-MHCN falls to the

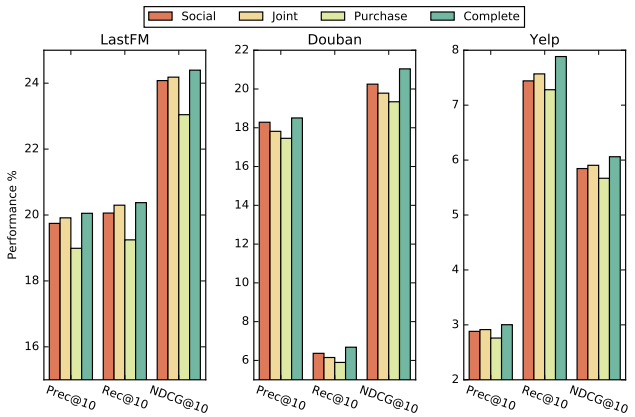

Figure 5: Contributions of each channel on different datasets.

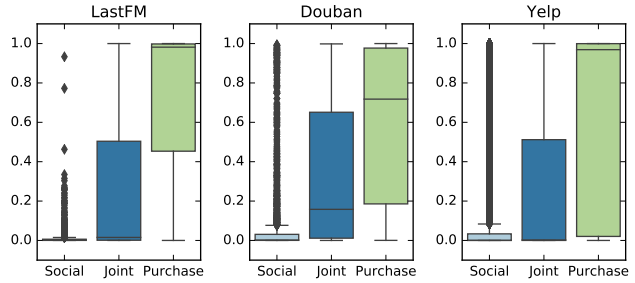

Figure 6: The distributions of the attention weights on different datasets.

level of LightGCN shown in Table 3. By contrast, removing *Social channel* or *Joint channel* would not have such a large impact on the final performance. Comparing *Social channel* with *Joint channel*, we can observe that the former contributes slightly more on LastFM and Yelp, while the latter, in terms of the performance contribution, is more important on Douban.

To further investigate the contribution of each channel when they are all employed, we visualize the attention scores learned along with other model parameters, and draw a box plot to display the distributions of the attention weights. According to Fig. 6, we can observe that, for the large majority of users in LastFM, *Social channel* has limited influence on the comprehensive user representations. In line with the conclusions from Fig. 5, *Purchase channel* plays the most important role in shaping the comprehensive user representations. The importance of *Joint channel* falls between the other two. The possible reason could be that, social relations are usually noisy and the users who are only socially connected might not always share similar preferences.

4.3.2 Investigation of Self-supervised Task. To investigate the effectiveness of the hierarchical mutual information maximization (MIM), we break this procedure into two parts: local MIM between the user and user-centered sub-hypergraph, and global MIM between the user-centered sub-hypergraph and hypergraph. We then run MHCN with either of these two to observe the performance changes. We also compare hierarchical MIM with the node-graph

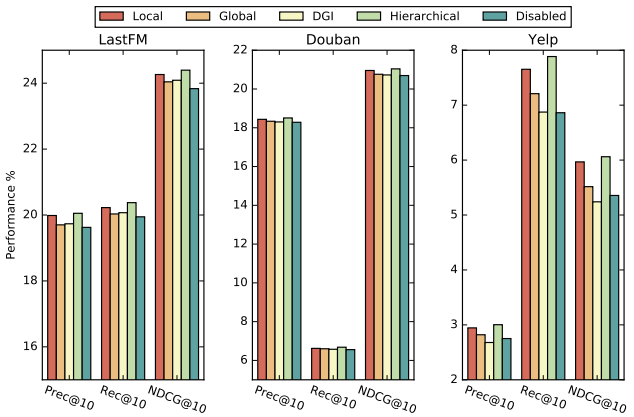

Figure 7: Investigation of Hierarchical Mutual Information Maximization on different datasets.

MIM used in DGI to validate the rationality of our design. We implement DGI by referring to the original paper [37]. The results are illustrated in Fig. 7, and we use *Disabled* to denote the vanilla MHCN. Unlike the bars in Fig. 6, each bar in Fig. 7 represents the case where only the corresponding module is used. As can be seen, hierarchical MIM shows the best performance while local MIM achieves the second best performance. By contrast, global MIM contributes less but it still shows better performance on Douban Yelp when compared with DGI. Actually, DGI almost rarely contributes on the latter two datasets and we can hardly find a proper parameter that can make it compatible with our task. On some metrics, training MHCN with DGI even lowers the performance. According to these results, we can draw a conclusion that the self-supervised task is effective and our intuition for hierarchical mutual information maximization is more reasonable compared with the node-graph MIM in DGI.

4.4 Parameter Sensitivity Analysis

In this section, we investigate the sensitivity of β and L.

As we adopt the *primary & auxiliary* paradigm, to avoid the negative interference from the auxiliary task in gradient propagating, we can only choose small values for β. We search the proper value in a small interval and empirically set it from 0.001 to 0.5. We then start our attempts from 0.001, and proceed by gradually increasing the step size. Here we report the performance of S^2-MHCN with eight representative β values $\{0.001, 0.005, 0.01, 0.02, 0.05, 0.1, 0.5\}$. As can be seen in Fig. 8, with the increase of the value of β, the performance of S^2-MHCN on all the datasets rises. After reaching the peak when β is 0.01 on all the datasets, it steadily declines. According to Fig. 8, we can draw a conclusion that even a very small β can promote the recommendation task, while a larger β would mislead it. The benefits brought by the self-supervised task could be easily neutralized and the recommendation task is sensitive to the magnitude of self-supervised task. So, choosing a small value is more likely to facilitate the primary task when there is little prior knowledge about the data distribution.

Finally, we investigate the influence of L to find the optimal depth for S^2-MHCN. We stack hypergraph convolutional layers from 1-layer to 5-layer setting. According to Fig. 9, the best performance

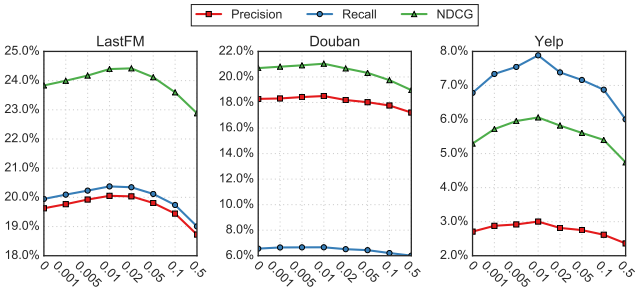

Figure 8: Influence of the magnitude of hierarchical MIM.

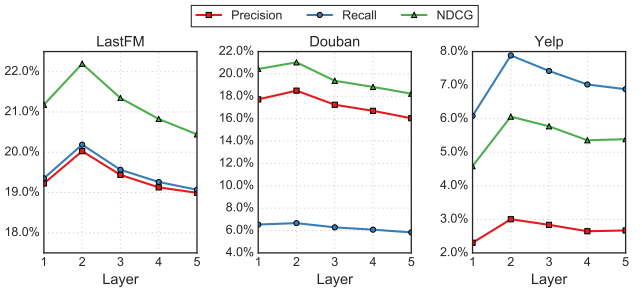

Figure 9: Influence of the depth of MHCN.

of S^2-MHCN is achieved when the depth of S^2-MHCN is 2. With the continuing increase of the number of layer, the performance of S^2-MHCN declines on all the datasets. Obviously, a shallow structure fits S^2-MHCN more. A possible reason is that S^2-MHCN aggregates high-order information from distant neighbors. As a result, it is more prone to encounter the over-smoothing problem with the increase of depth. This problem is also found in DHCF [16], which is based on hypergraph modeling as well. Considering the over-smoothed representations could be a pervasive problem in hypergraph convolutional network based models, we will work against it in the future.

5 CONCLUSION

Recently, GNN-based recommendation models have achieved great success in social recommendation. However, these methods simply model the user relations in social recommender systems as pairwise interactions, and neglect that real-world user interactions can be high-order. Hypergraph provides a natural way to model high-order user relations, and its potential for social recommendation has not been fully exploited. In this paper,we fuse hypergraph modeling and graph neural networks and then propose a multi-channel hypergraph convolutional network (MHCN) which works on multiple motif-induced hypergraphs to improve social recommendation. To compensate for the aggregating loss in MHCN, we innovatively integrate self-supervised learning into the training of MHCN. The self-supervised task serves as the auxiliary task to improve the recommendation task by maximizing hierarchical mutual information between the user, user-centered sub-hypergraph, and hypergraph representations. The extensive experiments conducted on three public datasets verify the effectiveness of each component of MHCN, and also demonstrate its state-of-the-art performance.

ACKNOWLEDGMENT

This work was supported by ARC Discovery Project (Grant No. DP190101985 and DP170103954). Jundong Li is supported by National Science Foundation (NSF) under grant No. 2006844.

REFERENCES

[1] Philip Bachman, R Devon Hjelm, and William Buchwalter. 2019. Learning representations by maximizing mutual information across views. In *Advances in Neural Information Processing Systems*. 15535–15545.
[2] Song Bai, Feihu Zhang, and Philip HS Torr. 2019. Hypergraph convolution and hypergraph attention. *arXiv preprint arXiv:1901.08150* (2019).
[3] Austin R Benson, David F Gleich, and Jure Leskovec. 2016. Higher-order organization of complex networks. *Science* 353, 6295 (2016), 163–166.
[4] Alain Bretto. 2013. Hypergraph theory. *An introduction. Mathematical Engineering. Cham: Springer* (2013).
[5] Jiajun Bu, Shulong Tan, Chun Chen, Can Wang, Hao Wu, Lijun Zhang, and Xiaofei He. 2010. Music recommendation by unified hypergraph: combining social media information and music content. In *Proceedings of the 18th ACM international conference on Multimedia*. 391–400.
[6] Lei Chen, Le Wu, Richang Hong, Kun Zhang, and Meng Wang. 2020. Revisiting Graph Based Collaborative Filtering: A Linear Residual Graph Convolutional Network Approach. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 34. 27–34.
[7] Robert B Cialdini and Noah J Goldstein. 2004. Social influence: Compliance and conformity. *Annu. Rev. Psychol.* 55 (2004), 591–621.
[8] Yann N Dauphin, Angela Fan, Michael Auli, and David Grangier. 2017. Language modeling with gated convolutional networks. In *International conference on machine learning*. 933–941.
[9] Wenqi Fan, Yao Ma, Qing Li, Yuan He, Eric Zhao, Jiliang Tang, and Dawei Yin. 2019. Graph neural networks for social recommendation. In *The World Wide Web Conference*. 417–426.
[10] Yifan Feng, Haoxuan You, Zizhao Zhang, Rongrong Ji, and Yue Gao. 2019. Hypergraph neural networks. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 33. 3558–3565.
[11] Min Gao, Junwei Zhang, Junliang Yu, Jundong Li, Junhao Wen, and Qingyu Xiong. 2020. Recommender Systems Based on Generative Adversarial Networks: A Problem-Driven Perspective. *arXiv preprint arXiv:2003.02474* (2020).
[12] Guibing Guo, Jie Zhang, and Neil Yorke-Smith. 2015. TrustSVD: collaborative filtering with both the explicit and implicit influence of user trust and of item ratings. In *Twenty-Ninth AAAI Conference on Artificial Intelligence*. 123–129.
[13] Lei Guo, Hongzhi Yin, Qinyong Wang, Tong Chen, Alexander Zhou, and Nguyen Quoc Viet Hung. 2019. Streaming session-based recommendation. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. 1569–1577.
[14] Xiangnan He, Kuan Deng, Xiang Wang, Yan Li, Yong-Dong Zhang, and Meng Wang. 2020. LightGCN: Simplifying and Powering Graph Convolution Network for Recommendation. In *Proceedings of the 43rd International ACM SIGIR conference on research and development in Information Retrieval*. ACM, 639–648.
[15] R Devon Hjelm, Alex Fedorov, Samuel Lavoie-Marchildon, Karan Grewal, Phil Bachman, Adam Trischler, and Yoshua Bengio. 2018. Learning deep representations by mutual information estimation and maximization. *arXiv preprint arXiv:1808.06670* (2018).
[16] Shuyi Ji, Yifan Feng, Rongrong Ji, Xibin Zhao, Wanwan Tang, and Yue Gao. 2020. Dual Channel Hypergraph Collaborative Filtering. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. 2020–2029.
[17] Wei Jin, Tyler Derr, Haochen Liu, Yiqi Wang, Suhang Wang, Zitao Liu, and Jiliang Tang. 2020. Self-supervised learning on graphs: Deep insights and new direction. *arXiv preprint arXiv:2006.10141* (2020).
[18] Mete Kemertas, Leila Pishdad, Konstantinos G Derpanis, and Afsaneh Fazly. 2020. RankMI: A Mutual Information Maximizing Ranking Loss. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. 14362–14371.
[19] Siwei Liu, Iadh Ounis, Craig Macdonald, and Zaiqiao Meng. 2020. A Heterogeneous Graph Neural Model for Cold-Start Recommendation. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*. 2029–2032.
[20] Hao Ma, Irwin King, and Michael R Lyu. 2009. Learning to recommend with social trust ensemble. In *International ACM SIGIR Conference on Research and Development in Information Retrieval*. 203–210.
[21] Hao Ma, Michael R. Lyu, and Irwin King. 2009. Learning to recommend with trust and distrust relationships. In *ACM Conference on Recommender Systems*.

189–196.

[22] Hao Ma, Haixuan Yang, Michael R Lyu, and Irwin King. 2008. Sorec: social recommendation using probabilistic matrix factorization. In *Proceedings of the 17th ACM conference on Information and knowledge management*. ACM, 931–940.

[23] Hao Ma, Dengyong Zhou, Chao Liu, Michael R Lyu, and Irwin King. 2011. Recommender systems with social regularization. In *Proceedings of the fourth ACM international conference on Web search and data mining*. ACM, 287–296.

[24] Jianxin Ma, Chang Zhou, Hongxia Yang, Peng Cui, Xin Wang, and Wenwu Zhu. 2020. Disentangled Self-Supervision in Sequential Recommenders. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. 483–491.

[25] Miller Mcpherson, Lynn Smithlovin, and James M Cook. 2001. Birds of a feather: Homophily in social networks. *Annual Review of Sociology* 27, 1 (2001), 415–444.

[26] Ron Milo, Shai Shen-Orr, Shalev Itzkovitz, Nadav Kashtan, Dmitri Chklovskii, and Uri Alon. 2002. Network motifs: simple building blocks of complex networks. *Science* 298, 5594 (2002), 824–827.

[27] Aaron van den Oord, Yazhe Li, and Oriol Vinyals. 2018. Representation learning with contrastive predictive coding. *arXiv preprint arXiv:1807.03748* (2018).

[28] Zhen Peng, Wenbing Huang, Minnan Luo, Qinghua Zheng, Yu Rong, Tingyang Xu, and Junzhou Huang. 2020. Graph Representation Learning via Graphical Mutual Information Maximization. In *Proceedings of The Web Conference 2020*. 259–270.

[29] Jiezhong Qiu, Qibin Chen, Yuxiao Dong, Jing Zhang, Hongxia Yang, Ming Ding, Kuansan Wang, and Jie Tang. 2020. GCC: Graph Contrastive Coding for Graph Neural Network Pre-Training. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. 1150–1160.

[30] Steffen Rendle, Christoph Freudenthaler, Zeno Gantner, and Lars Schmidt-Thieme. 2009. BPR: Bayesian personalized ranking from implicit feedback. In *Proceedings of the twenty-fifth conference on uncertainty in artificial intelligence*. AUAI Press, 452–461.

[31] Aravind Sankar, Junting Wang, Adit Krishnan, and Hari Sundaram. 2020. Beyond Localized Graph Neural Networks: An Attributed Motif Regularization Framework. *arXiv preprint arXiv:2009.05197* (2020).

[32] Aravind Sankar, Yanhong Wu, Yuhang Wu, Wei Zhang, Hao Yang, and Hari Sundaram. 2020. GroupIM: A Mutual Information Maximization Framework for Neural Group Recommendation. *arXiv preprint arXiv:2006.03736* (2020).

[33] Badrul Munir Sarwar. 2001. Sparsity, scalability, and distribution in recommender systems. (2001).

[34] Weiping Song, Zhiping Xiao, Yifan Wang, Laurent Charlin, Ming Zhang, and Jian Tang. 2019. Session-based social recommendation via dynamic graph attention networks. In *Proceedings of the Twelfth ACM International Conference on Web Search and Data Mining*. 555–563.

[35] Ke Sun, Zhouchen Lin, and Zhanxing Zhu. 2019. Multi-Stage Self-Supervised Learning for Graph Convolutional Networks on Graphs with Few Labels. *arXiv preprint arXiv:1902.11038* (2019).

[36] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *Advances in neural information processing systems*. 5998–6008.

[37] Petar Velickovic, William Fedus, William L Hamilton, Pietro Liò, Yoshua Bengio, and R Devon Hjelm. 2019. Deep Graph Infomax.. In *ICLR (Poster)*.

[38] Jianling Wang, Kaize Ding, Liangjie Hong, Huan Liu, and James Caverlee. 2020. Next-item Recommendation with Sequential Hypergraphs. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*. 1101–1110.

[39] Le Wu, Yong Ge, Qi Liu, Enhong Chen, Richang Hong, Junping Du, and Meng Wang. 2017. Modeling the evolution of users' preferences and social links in social networking services. *IEEE Transactions on Knowledge and Data Engineering* 29, 6 (2017), 1240–1253.

[40] Le Wu, Junwei Li, Peijie Sun, Yong Ge, and Meng Wang. 2020. DiffNet++: A Neural Influence and Interest Diffusion Network for Social Recommendation. *arXiv preprint arXiv:2002.00844* (2020).

[41] Le Wu, Peijie Sun, Yanjie Fu, Richang Hong, Xiting Wang, and Meng Wang. 2019. A Neural Influence Diffusion Model for Social Recommendation. *CoRR* abs/1904.10322 (2019).

[42] Qitian Wu, Hengrui Zhang, Xiaofeng Gao, Peng He, Paul Weng, Han Gao, and Guihai Chen. 2019. Dual graph attention networks for deep latent representation of multifaceted social effects in recommender systems. In *The World Wide Web Conference*. 2091–2102.

[43] Zonghan Wu, Shirui Pan, Fengwen Chen, Guodong Long, Chengqi Zhang, and S Yu Philip. 2020. A comprehensive survey on graph neural networks. *IEEE Transactions on Neural Networks and Learning Systems* (2020).

[44] Xin Xia, Hongzhi Yin, Junliang Yu, Qinyong Wang, Lizhen Cui, and Xiangliang Zhang. 2020. Self-Supervised Hypergraph Convolutional Networks for Session-based Recommendation. *arXiv preprint arXiv:2012.06852* (2020).

[45] Xin Xin, Alexandros Karatzoglou, Ioannis Arapakis, and Joemon M Jose. 2020. Self-Supervised Reinforcement Learning forRecommender Systems. *arXiv preprint arXiv:2006.05779* (2020).

[46] Bo Yang, Yu Lei, Jiming Liu, and Wenjie Li. 2017. Social collaborative filtering by trust. *IEEE transactions on pattern analysis and machine intelligence* 39, 8 (2017), 1633–1647.

[47] Dingqi Yang, Bingqing Qu, Jie Yang, and Philippe Cudre-Mauroux. 2019. Revisiting user mobility and social relationships in lbsns: a hypergraph embedding approach. In *The World Wide Web Conference*. 2147–2157.

[48] Hongzhi Yin, Zhiting Hu, Xiaofang Zhou, Hao Wang, Kai Zheng, Quoc Viet Hung Nguyen, and Shazia Sadiq. 2016. Discovering interpretable geo-social communities for user behavior prediction. In *Data Engineering (ICDE), 2016 IEEE 32nd International Conference on*. IEEE, 942–953.

[49] Hongzhi Yin, Qinyong Wang, Kai Zheng, Zhixu Li, Jiali Yang, and Xiaofang Zhou. 2019. Social influence-based group representation learning for group recommendation. In *2019 IEEE 35th International Conference on Data Engineering (ICDE)*. IEEE, 566–577.

[50] Hongzhi Yin, Qinyong Wang, Kai Zheng, Zhixu Li, and Xiaofang Zhou. 2020. Overcoming Data Sparsity in Group Recommendation. *IEEE Transactions on Knowledge and Data Engineering* (2020).

[51] Hongzhi Yin, Xiaofang Zhou, Bin Cui, Hao Wang, Kai Zheng, and Quoc Viet Hung Nguyen. 2016. Adapting to User Interest Drift for POI Recommendation. *IEEE Transactions on Knowledge and Data Engineering* 28, 10 (2016), 2566–2581.

[52] Hongzhi Yin, Lei Zou, Quoc Viet Hung Nguyen, Zi Huang, and Xiaofang Zhou. 2018. Joint event-partner recommendation in event-based social networks. In *2018 IEEE 34th International Conference on Data Engineering (ICDE)*. IEEE, 929–940.

[53] Yuning You, Tianlong Chen, Yongduo Sui, Ting Chen, Zhangyang Wang, and Yang Shen. 2020. Graph Contrastive Learning with Augmentations. *Advances in Neural Information Processing Systems* 33 (2020).

[54] Dingjun Yu, Hanli Wang, Peiqiu Chen, and Zhihua Wei. 2014. Mixed pooling for convolutional neural networks. In *International conference on rough sets and knowledge technology*. Springer, 364–375.

[55] Junliang Yu, Min Gao, Jundong Li, Hongzhi Yin, and Huan Liu. 2018. Adaptive Implicit Friends Identification over Heterogeneous Network for Social Recommendation. In *Proceedings of the 27th ACM International Conference on Information and Knowledge Management*. ACM, 357–366.

[56] Junliang Yu, Min Gao, Wenge Rong, Wentao Li, Qingyu Xiong, and Junhao Wen. 2017. Hybrid attacks on model-based social recommender systems. *Physica A: Statistical Mechanics and its Applications* 483 (2017), 171–181.

[57] Junliang Yu, Min Gao, Hongzhi Yin, Jundong Li, Chongming Gao, and Qinyong Wang. 2019. Generating reliable friends via adversarial training to improve social recommendation. In *2019 IEEE International Conference on Data Mining (ICDM)*. IEEE, 768–777.

[58] Junliang Yu, Hongzhi Yin, Jundong Li, Min Gao, Zi Huang, and Lizhen Cui. 2020. Enhance Social Recommendation with Adversarial Graph Convolutional Networks. *arXiv preprint arXiv:2004.02340* (2020).

[59] Xiaohua Zhai, Avital Oliver, Alexander Kolesnikov, and Lucas Beyer. 2019. S4l: Self-supervised semi-supervised learning. In *Proceedings of the IEEE international conference on computer vision*. 1476–1485.

[60] Huan Zhao, Xiaogang Xu, Yangqiu Song, Dik Lun Lee, Zhao Chen, and Han Gao. 2018. Ranking Users in Social Networks With Higher-Order Structures.. In *AAAI*. 232–240.

[61] Tong Zhao, Julian McAuley, and Irwin King. 2014. Leveraging social connections to improve personalized ranking for collaborative filtering. In *Proceedings of the 23rd ACM International Conference on Conference on Information and Knowledge Management*. ACM, 261–270.

[62] Xiaoyao Zheng, Yonglong Luo, Liping Sun, Xintao Ding, and Ji Zhang. 2018. A novel social network hybrid recommender system based on hypergraph topologic structure. *World Wide Web* 21, 4 (2018), 985–1013.

[63] Jie Zhou, Ganqu Cui, Zhengyan Zhang, Cheng Yang, Zhiyuan Liu, Lifeng Wang, Changcheng Li, and Maosong Sun. 2018. Graph neural networks: A review of methods and applications. *arXiv preprint arXiv:1812.08434* (2018).

[64] Kun Zhou, Hui Wang, Wayne Xin Zhao, Yutao Zhu, Sirui Wang, Fuzheng Zhang, Zhongyuan Wang, and Ji-Rong Wen. 2020. Sˆ 3-Rec: Self-Supervised Learning for Sequential Recommendation with Mutual Information Maximization. *arXiv preprint arXiv:2008.07873* (2020).